Parallelization Strategies for DLRM Embedding Bag Operator on AMD CPUs

Krishnakumar Nair, *Advanced Micro Devices, Santa Clara, CA, 95054, USA*
Avinash-Chandra Pandey and Siddappa Karabannavar, *Advanced Micro Devices, Bengaluru, 560048, India*
Meena Arunachalam, *Advanced Micro Devices, Santa Clara, CA, 95054, USA*
John Kalamatianos, *Advanced Micro Devices, Boxborough, MA, 01719, USA*
Varun Agrawal and Saurabh Gupta, *Advanced Micro Devices, Bengaluru, 560048, India*
Ashish Sirasao and Elliott Delaye, *Advanced Micro Devices, Santa Clara, CA, 95054, USA*
Steve Reinhardt, *Advanced Micro Devices, Redmond, WA, 98052, USA*
Rajesh Vivekanandham, *Advanced Micro Devices, Bengaluru, 560048, India*
Ralph Wittig and Vinod Kathail, *Advanced Micro Devices, San Jose, CA, 95124, USA*
Padmini Gopalakrishnan, *Advanced Micro Devices, Hyderabad, 500081, India*
Satyaprakash Pareek, *Advanced Micro Devices, San Jose, CA, 95124, USA*
Rishabh Jain, Mahmut Taylan Kandemir, Jun-Liang Lin, Gulsum Gudukbay Akbulut, and Chita R. Das, *Pennsylvania State University, University Park, PA, 16802, USA*

Deep learning recommendation models (DLRMs) are deployed extensively to support personalized recommendations and consume a large fraction of artificial intelligence (AI) cycles in modern datacenters with embedding stage being a critical component. Modern CPUs execute a lot of DLRM cycles because they are cost effective compared to GPUs and other accelerators. Our paper addresses key bottlenecks in accelerating the embedding stage on CPUs. Specifically, this work 1) explores novel threading schemes that parallelize embedding bag, 2) pushes the envelope on realized bandwidth by improving data reuse in caches, and 3) studies the impact of parallelization on load imbalance. The new embedding bag kernels have been prototyped in the ZenDNN software stack. When put together, our work on fourth generation EPYC processors achieve up to 9.9x improvement in embedding bag performance over state-of-the-art implementations, and improve realized bandwidth of up to 5.7x over DDR bandwidth.

Recommendation systems are an important class of machine learning (ML) models that provide personalized recommendations. These models, also called deep learning recommendation models (DLRMs),[4,6,10] are deployed widely across the industry. Optimizing DLRM remains a key objective for the foreseeable future because recommendation systems are used in ads generation as well as in user base expansion. A critical prelude to any DLRM optimization is a thorough "characterization" of DLRM workloads on target compute infrastructures.

As an example, the DLRM model published by Meta[10] is depicted in Figure 1(a). This model employs multilayer perceptrons (MLPs), embeddings, and an interaction layer, each having unique characteristics.

0272-1732 © 2024 IEEE
Digital Object Identifier 10.1109/MM.2024.3423785
Date of publication 5 August 2024; date of current version 11 December 2024.

The top and bottom MLPs are compute-dominated. In contrast, the embedding stage is memory bandwidth- and memory capacity-dominated. Even though efforts of tolerating the overhead of the embedding stage, while parallelizing the MLPs, have been successful,[6,7] a significant portion of DLRM inference cycles is still spent in the embedding stage. Hence, innovations in improving the performance of embedding stage can have a nontrivial impact on the overall DLRM inference latency. This in turn can lead to improvements in queries-per-second while meeting quality-of-service requirements.

MOTIVATION

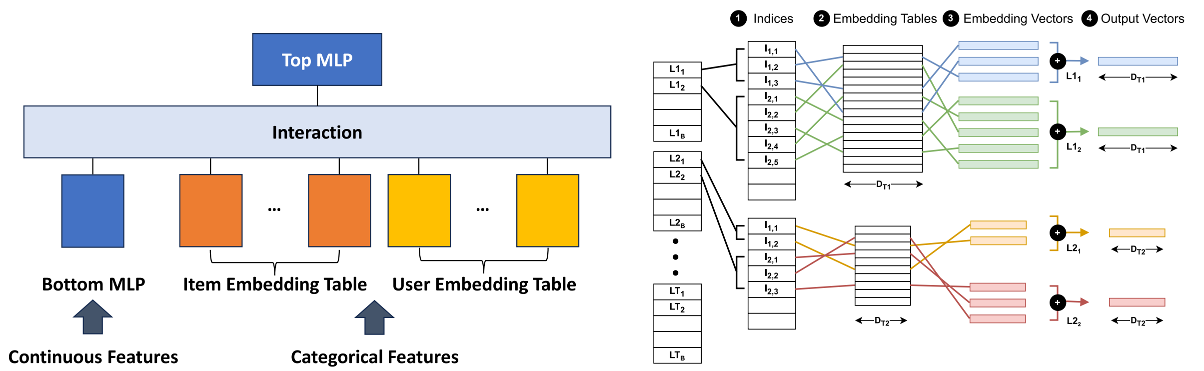
Our goal is to enable scalable inference with DLRMs that utilize very large, sparse embedding tables. Our proposed approach is motivated by two observations:

- First, while DLRM can be accelerated by GPUs or custom accelerators, CPUs can be a more "cost-effective" platform where it is easier to deploy and optimize DLRM, compared to other target architectures.
- Second, it is known that DLRM exhibits a large degree of data reuse when accessing the embedding tables and the majority of the table indices follow a Zipf's law distribution.[3] Preliminary data for leveraging this data reuse for select hot tables has been shown to provide good performance for embedding operations.[7] However, the embedding stage remains a memory-bound kernel. The state-of-the-art embedding bag parallelization method on CPU multicores is called *batch threading* (BT) and parallelizes accesses to each embedding table across all available threads (more on this later). The current threading strategy of BT, shown in Figure 2(a), reveals that the achieved bandwidth is at least 43.6% lower than the peak last level cache (LLC) bandwidth and in some cases 40% lower than the peak DDR bandwidth.

CONTRIBUTIONS

Motivated by the previous observations, this article makes the following novel contributions:

- We leverage the cache hierarchy and chiplet-based architecture found in state-of-the-art, server-grade CPU multicores to propose different parallelization strategies for the embedding bag operator, which enable scaling and improve data processing rate. Our proposed parallelization strategies, referred to as *table threading* (TT) and *hierarchical threading* (HT), leverage the high data reuse in embedding tables to improve the utilization of L1, L2, and L3 (LLC) caches, rather than relying on the available DDR bandwidth.
- We compare our proposed parallelization strategies on state-of-the-art CPU multicores, and explain why some parallelization strategies perform better than others. Specifically, the experimental evaluations indicate that our proposed embedding bag parallelization scheme achieves, on 4th Gen EPYC processors, up to 9.9x improvement in embedding performance over the state-of-the-art BT parallelization strategy.



(a) (b)

FIGURE 1. (a) DLRM architecture. MLPs are compute-bound. Embedding tables are memory bandwidth- and memory capacity-bound. (b) Embedding bag operator workflow.

NOVELTY

Prior work on accelerating DLRM Inference on CPU multicores[7] proposed parallelization at coarse levels of granularity: 1) executing bottom MLP in parallel to the embedding stage and 2) executing an entire batch per CPU core for the embedding stage. In this article, we focus on finer granularity parallelization strategies that better exploit the underlying architecture: 1) we propose novel parallelization schemes that exploit a chiplet-based multicore architecture consisting of core complexes, each with a shared LLC, by pinning the per-table or per table-group memory traffic onto CPU cores or Core complexes; 2) we study the scaling of the embedding stage on production datasets, which combine heterogeneous aspects such as nonuniform table sizes, varying number of lookups per user, and different row access distribution across tables; 3) we identify embedding bag configurations (based on batch size, table size, embedding dimension, and row reuse distribution), scaling when running on state-of-the-art CPU multicores; and finally, 4) we identify the performance scaling impact of 3D stacked LLC on the embedding bag operator.

BACKGROUND

Understanding Embedding Bag Operator

A typical embedding bag is illustrated in Figure 1(b). Each embedding operator does a series of vector reads into a table. The width of each vector is called *embedding dimension*. A *pooling factor* number of vectors are *pooled* together to form a single *output* vector. Multiple such embedding operations are batched together for each table (size of the pooling factor can vary across batches). This operation is then repeated across multiple tables. The table size varies.[9]

Current implementations of the embedding bag operator process the work per table in parallel, but invoke the operator across tables *sequentially*. Therefore, the time required to complete the embedding bag kernel is the *sum* of the per-table processing time.

Target Architectures

AMD "Genoa"

"Genoa" is based on the successful multichip module (MCM)-based Chiplet architecture system-on-a-chip.

AMD "Genoa-X" – ("Genoa" + V-Cache)

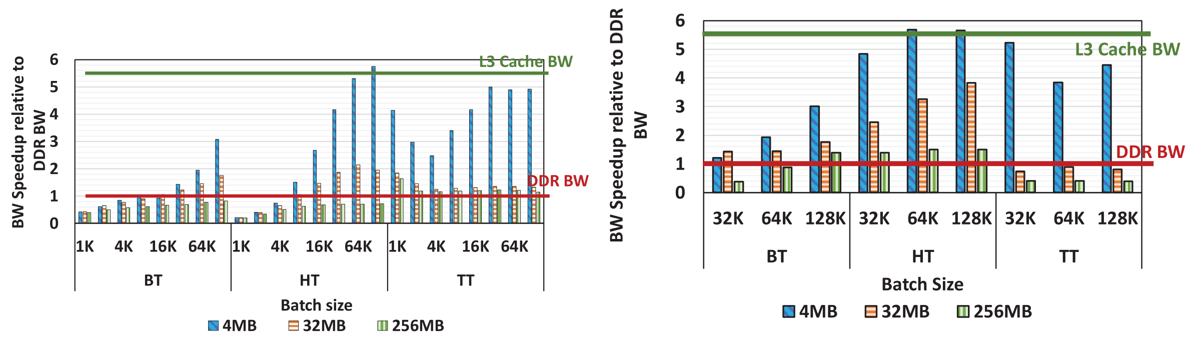
"Genoa-X" is available with a 3D stacked L3 cache (called "V-Cache") that provides an additional 64-MB capacity per core complex for a total of 96-MB L3 per core complex and 1.12 GB of L3 per socket.[1]

LOAD IMBALANCE IN EMBEDDING BAG

For a given model, the time to process a table depends on two factors: the number of lookups per input and the latency per lookup. Thus, the time per table could vary.

In data sets published by Meta,[9] we observe significant variations in DLRM model properties such as the number of lookups per input or table sizes. As mentioned previously, the current implementations of embedding bag kernel depend on the properties of different tables because each table is processed independently. The proposed parallelization strategies (discussed in the next section) mitigate the performance variability caused by the *nonuniformity* in table properties.

Meta's DLRM production traces show significant *nonuniformity* across parameters such as row sizes,



(a) (b)

FIGURE 2. (a) Realized bandwidth relative to DDR bandwidth for "Genoa." (b) Realized bandwidth relative to DDR bandwidth for "Genoa-X."

pooling factors, memory access patterns, and time taken per table.

The DLRM model skews heavily toward tables with row sizes between 100 and five million, which translates to varying memory capacity requirements per table. Other contributing factors that influence work per table are the number of lookups per user and table row locality of access.[7]

Figure 3 shows the time taken by each table in descending order. The range is between 40 µs to 2292 µs, with an average time of 301 µs. The time to process these tables on a single core can vary by up to 57x.

PARALLELIZATION STRATEGIES FOR EMBEDDING BAG OPERATOR

We propose two novel parallelization strategies (called TT and HT) for the embedding bag operator. The default embedding bag parallelization strategy for CPU-based DLRM systems is referred to as *batch threading*.[3] We also present a qualitative analysis of the three strategies [illustrated in Figure 4(a)–(c)] and explain the scenarios under which each strategy is expected to outperform the others. An experimental evaluation/comparison of these strategies is presented in the "Experimental Evaluation" section.

BT

The default strategy, BT, is shown in Figure 4(a), both pictorially as well as in a pseudo-code format. BT maps batches within a table to all hardware contexts (threads). Subsequent tables are scheduled after all batches in the current table complete. Under BT, tables are processed sequentially as shown in pseudocode in Figure 4(a). Since, under BT (which is the scheme employed in ZenDNN and vanilla PyTorch), the work per table is spread across all cores, shared hot table rows accessed by different offsets (and thus threads) can be shared via the shared L3 caches. In the target chiplet-based CPU multicore architecture, this may lead to high cache-to-cache traffic both within the same and across core complexes and may not utilize well the on-die cache capacity. In reality, table sizes vary; thus, when processing small tables, caches (as well as other on-chip resources) may not be well utilized. Furthermore, since the work per offset varies, there can be load imbalance across the threads. Since a batch-size number of threads are created, where each offset represents a very small amount of work, threads would observe significant runtime overheads (e.g., from fork/join system calls). We next discuss new parallelization schemes which attempt to overcome these problems.

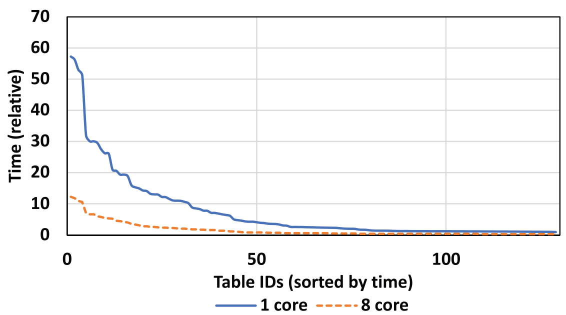


FIGURE 3. Embedding table latency for different tables.

TT

TT assigns the work of each table to a single thread. The work of a table is assigned to a core only after the work of the previous table is complete. Hot table rows accessed by different offsets are now accessed via the private core caches (L1 and L2). However, shared caches (L3) store table rows that do not fit in private caches and can experience contention among threads, depending on table working set size. Moreover, with TT the overhead of thread fork/join is amortized over more work, and there is no duplication of data across private caches.

However, the amount of work per table and table size varies (see the "Load Imbalance in Embedding Bag" section) which can translate to poor load balancing across threads with TT, leading to lower core utilization. In the case where the number of tables is the same as the number of threads, the overall execution time is determined by the processing time of the slowest table.

HT

BT is expected to have good load balance; but can suffer from cache-to-cache traffic and low cache capacity utilization. In contrast, TT is expected to have better cache performance but higher load imbalance. HT strikes a balance between these two parallelization strategies. Under HT, the tables are distributed across chiplets (called CCDs[a]) and the threads (cores) in a CCD operate—in parallel—on the tables assigned to that CCD, one table at a time. In other words, HT shares table hot rows both via private caches (as in TT) and shared (L3) cache (as in BT). Unlike BT, it does not require cross-CCD traffic for shared hot rows because all threads for the same table are mapped to the same

[a]In AMD architectures, a CCD is a cluster of eight CPU cores that share an L3 cache. Depending on the system, an AMD CPU can have one or more active CCDs per physical processor.

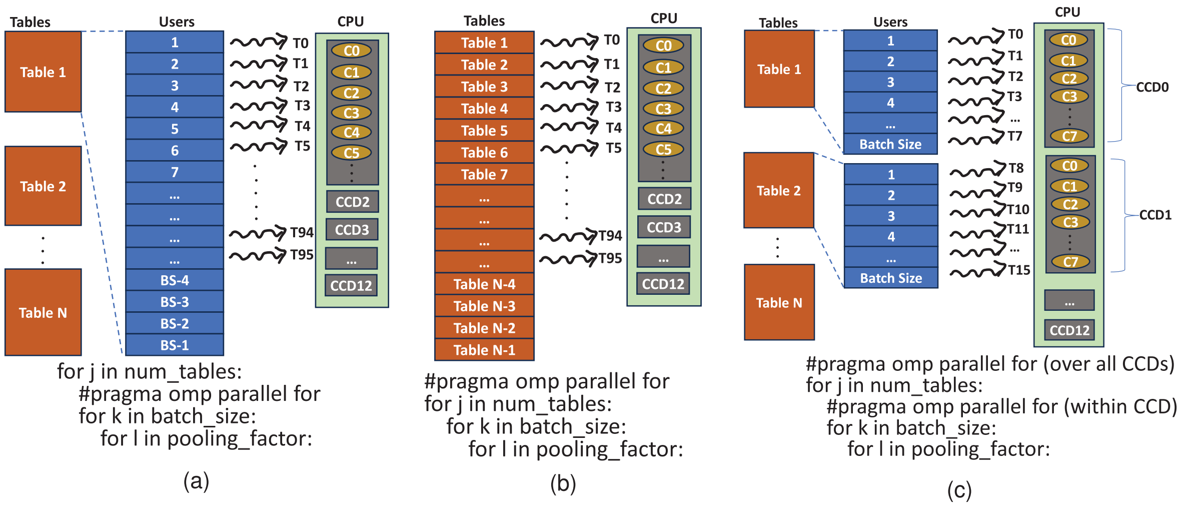


FIGURE 4. Threading strategies. (a) Batch threading, (b) Table threading, (c) Hierarchical threading.

CCD. If necessary, the table assignment granularity can be less than a CCD, e.g., two cores or four cores.

HT can enable good load balance and good cache performance by careful *mapping* of work per table to the appropriate number of cores (threads).

EXPERIMENTAL EVALUATION

In this section, we experimentally evaluate the three embedding bag parallelization strategies.

Methodology

Datasets

Our experimental evaluation includes *both* synthetic traces and real-life traces, grouped into two categories: "homogeneous" and "heterogeneous." The homogeneous datasets possess uniform properties across tables such as table size, batch size, pooling factor, α (the exponent parameter), and embedding dimension for a particular performance run. Since embedding lookup indices are known to follow Zipf's distribution,[2,3,9] we follow the methodology used in FBGEMM,[3] by injecting indices generated from Zipf's distribution for measurement.

In addition to these datasets, we also used *representative* datasets provided by Meta in their DLRM dataset repository.[9] This dataset is referred to as *heterogeneous* because of the nonuniform table properties such as number of rows, lookups per user, and memory access patterns.

Metrics

As discussed in the "Load Imbalance in Embedding Bag" section, embedding bag is a memory intensive kernel. Therefore, we believe *bandwidth* is a representative metric for its performance. With every batch, the total work can be thought as the amount of data to be processed. Bandwidth can be calculated as a product of batch size, number of tables, pooling factor, embedding dimension, and precision of embedding values. Note that the realized bandwidth (or bandwidth, for short) can be calculated using the formula (amount of data read)/(batch latency) which is interpreted as the *rate* at which *amount of data* is consumed, thus having the unit GBps. This metric allows us to compare the rate of work across different model configurations.

BANDWIDTH CAN BE CALCULATED AS A PRODUCT OF BATCH SIZE, NUMBER OF TABLES, POOLING FACTOR, EMBEDDING DIMENSION, AND PRECISION OF EMBEDDING VALUES.

Results

We start by presenting, in Figure 2(a), the performance of the parallelization schemes assuming a Zipf-based table access distribution. All results are *normalized* to BT. We observe that, for table sizes that fit in the L3 (table size of 4 MB), TT and HT outperform BT (4.5x and 3x, respectively) because both TT and HT process more work (tables) in parallel than BT while memory traffic

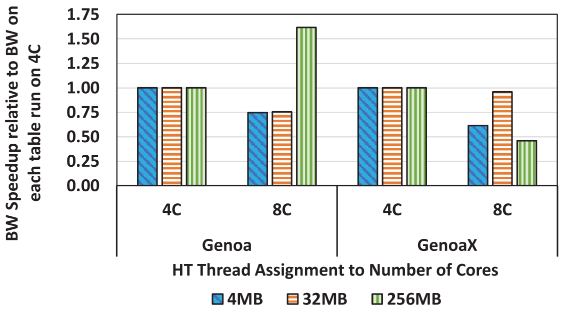


FIGURE 5. HT performance when each thread is scheduled on four cores (4 C) versus eight cores (8 C) for "Genoa" and "Genoa-X." All numbers normalized to the 4 C case per table size. Number of tables is 192.

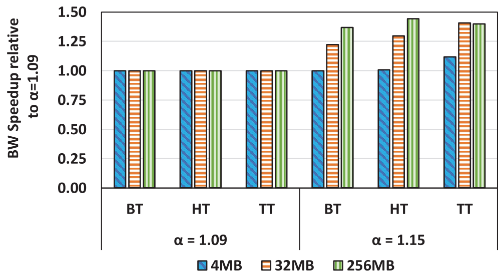


FIGURE 6. Parallelization scheme performance with α (α = 1.15 versus α = 1.09) for different table sizes

is not high enough (due to table row locality) to significantly stall execution in the cores. TT outperforms HT for the same reason. We expect a similar trend for smaller table sizes that fit in L3. For medium sized table size of 32 M, HT outperforms BT and TT. Overall performance is lower than that of 4 MB tables because of more capacity/conflict L3 misses due to the increased data working set per thread. HT has better performance than TT because of the lower data working set size per CCD, which leads to less memory traffic. Finally, for large enough tables (e.g., table sizes of 256 MB), TT outperforms both BT and HT.

ON THE OTHER HAND, WHEN THE WORKING SET PER CCD DOES NOT FIT IN THE L3 CACHE, THE PERFORMANCE IMPROVES DUE TO BETTER TABLE ROW REUSE ACROSS THE CACHES.

The supported memory level parallelism (MLP) per CCD is defined by the number of L3 miss status hold registers (MSHRs) and remains independent of the table size or number of tables assigned per CCD. If we assign sufficient table capacity or number of tables per CCD whose accessed data do not fit in the L3 but saturate MLP, the fastest parallelization scheme is the one that processes the most tables simultaneously. We observe from Figure 2(b) that HT outperforms both TT and BT even for large tables (256 MB). This is because, "Genoa-X"'s much larger cache capacity significantly reduces L3 capacity/conflict misses and prevents the CCD's MLP from saturating, allowing HT to perform better due to lower overall memory traffic.

We also studied the sensitivity of HT to the number of cores per CCD assigned per thread. Figure 5 shows results when assigning a thread to either half (4 C) or all cores (8 C) per CCD in both "Genoa" and "Genoa-X" for 192 tables. In the 4 C configuration, two tables are scheduled on the same CCD, increasing the work processing rate of HT compared to 8 C. Therefore, if the table size fits into L3 (e.g., table sizes of 4 MB and 32 MB), embedding bag becomes mostly compute-bound and its performance scales better with 4 C for both "Genoa" and "Genoa-X." When memory traffic dominates (e.g., 256-MB table size on "Genoa"), 8 C performs better because of the lack of inter-thread conflict L3 misses (observed in 4 C). Note that the inter-thread conflict L3 misses are much less of an issue in "Genoa-X" due to its much larger L3 capacity (number of cache sets), and as a result, 4 C performs better.

All results discussed so far use Zipf's distribution having an $\alpha = 1.09$. We also evaluated the parallelization schemes with higher table row reuse by using $\alpha = 1.15$. In Figure 6, we evaluate the performance change relative to $\alpha = 1.09$ on "Genoa." All values are *normalized* to $\alpha = 1.09$ for each scheme separately. When the working set per CCD fits into L3 (e.g., table sizes of 4 MB for all three schemes) there is either no or small performance change (TT which suffers from intertable interference in L3 sees small performance gain). On the other hand, when the working set per CCD does not fit in the L3 cache, the performance improves due to better table row reuse across the caches. TT does not benefit really large tables (e.g., 256 MB) because of very high intertable interference.

In Figure 2(a) and (b), we show the gains of our proposed threading schemes over DDR bandwidth versus BT on "Genoa" and "Genoa-X." On "Genoa," we realize 5.7x bandwidth relative to DDR using HT on table size of 4 MB for batch size of 64 K. For smaller batch size of 1 K and 2 K for a 4 MB table size, TT generates 3x and 2.5x bandwidth over DDR. HT performs well on "Genoa-X" with 32 MB and 256 MB tables. We see a realized bandwidth of 3.8x and 1.5x relative to DDR bandwidth for 32 MB and 256 MB tables, respectively, for a batch size of 128 K.

Also, recall from the "Load Imbalance in Embedding Bag" section that the work per table heavily varies which leads to large variation in execution times per core. This is also highlighted in Figure 3 which shows the distribution of time per table in the heterogeneous dataset with HT. The execution time ratio of the largest over smallest sized table is 4.7x lower, when embedding bag is executed with eight cores.

RELATED WORK

DLRM[10] has gained a lot of attention in industry since it was first proposed by Facebook in 2019. Several works have since been proposed to accelerate DLRM training or inference.[2,4,5,6,7,8]

For example,[7] explored prefetching, hyperthreading, and overlapping of computation with memory accesses, to accelerate DLRM inference on CPUs. In comparison, Ardestani et al.[2] accelerated DLRM inference by extending the memory hierarchy to storage class memory through software-defined memory.

Due to the extremely large size of embedding tables, many prior works concentrated on optimizing memory-intensive operators. Further efforts to enhance memory efficiency, including, Lee et al.[8] proposed embedding- or feature-based optimizations.

CONCLUDING REMARKS

General-purpose CPUs are ubiquitous in data centers and can be cost effective compared to GPUs. As a result, they consume a lot of DLRM inference cycles which in turn form a large fraction of AI cycles in data centers. The embedding stage of DLRM is limited by memory bandwidth and capacity. Table indices accessed in embedding stage follow Zipf's distribution which has high potential for reuse. AMD 4th Generation EPYC™ processors have large caches and high DDR capacity that can benefit the embedding stage. While prior DLRM optimizations on CPUs have focused on software prefetching and computation overlapping, this article proposed chiplet-aware parallelization techniques (TT and HT) that better utilize on-chip caches and scale DLRM inference on CPU multicores across the full core count. We demonstrate up to 9.9x speedup on homogeneous datasets based on Zipf's distribution and 2.95x speedup on heterogeneous datasets. We realize an effective bandwidth of up to 5.7x over DDR bandwidth. Our techniques have been integrated in the CPU-based ZenDNN library, making them a competitive platform for DLRM inference. Our work sets a new baseline for future research targeting DLRM inference.

ACKNOWLEDGMENTS

We thank Siddharth Rele, Zachary Blair, Matt Oulette, Srilatha Manne, and Greg Bolet for their contributions. Due to the constraint on the number of authors imposed by the Micro magazine publication, we are unable to add them as coauthors on the paper.

REFERENCES

1. "AMD Epyc 9684X CPU, code-named Genoa X." AMD. [Online]. Available: https://www.amd.com/en/products/cpu/amd-epyc-9684x
2. E. K. Ardestani et al., "Supporting massive DLRM inference through software defined memory," in *Proc. IEEE 42nd Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 302–312, doi: 10.1109/ICDCS54860.2022.00037.
3. Facebook. "Facebook generalized matrix multiplication." GitHub. [Online]. Available: https://github.com/pytorch/FBGEMM
4. U. Gupta et al., "DeepRecSys: A system for optimizing end-to-end at-scale neural recommendation inference," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit. (ISCA)*, Valencia, Spain, 2020, pp. 982–995, doi: 10.1109/ISCA45697.2020.00084.
5. U. Gupta et al., "RecPipe: Co-designing models and hardware to jointly optimize recommendation quality and performance," in *Proc. 54th Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, New York, NY, USA: ACM, 2021, pp. 870–884, doi: 10.1145/3466752.3480127.
6. U. Gupta et al., "The architectural implications of Facebook's DNN-based personalized recommendation," in *Proc. IEEE Int. Symp. High Perform. Comput. Architect. (HPCA)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 488–501, doi: 10.1109/HPCA47549.2020.00047.
7. R. Jain et al., "Optimizing CPU performance for recommendation systems at-scale," in *Proc. 50th Annu. Int. Symp. Comput. Archit. (ISCA)*, New York, NY, USA: ACM, 2023, pp. 1–15, doi: 10.1145/357937w1.3589112.
8. Y. Lee et al., "MERCI: Efficient embedding reduction on commodity hardware via sub-query memoization," in *Proc. 26th ACM Int. Conf. Archit. Support Program. Lang. Operating Syst. (ASPLOS)*, New York, NY, USA: ACM, 2021, pp. 302–313, doi: 10.1145/3445814.3446717.
9. Meta. "Embedding lookup Production dataset." [Online]. Available: GitHub. [Online]. Available: https://github.com/facebookresearch/dlrm_datasets

10. M. Naumov et al., “Deep learning recommendation model for personalization and recommendation systems,” 2019, *arXiv:1906.00091*.

KRISHNAKUMAR NAIR is a fellow software development engineer at Advanced Micro Devices, Santa Clara, CA, 95054, USA. Contact him at krishnakumar.nair@amd.com.

AVINASH-CHANDRA PANDEY is a software system design engineer at Advanced Micro Devices, Bengaluru, 560048, India. Contact him at avinash-chandra.pandey@amd.com.

SIDDAPPA KARABANNAVAR is a software system design engineer at Advanced Micro Devices, Bengaluru, 560048, India. Contact him at siddappa.karabannavar@amd.com.

MEENA ARUNACHALAM is a fellow systems design engineer at Advanced Micro Devices, Santa Clara, CA, 95054, USA. Contact her at meena.arunachalam@amd.com.

JOHN KALAMATIANOS is a fellow silicon design engineer at Advanced Micro Devices, Boxborough, MA, 01719, USA. Contact him at john.kalamatianos@amd.com.

VARUN AGRAWAL is a software system design engineer at Advanced Micro Devices, Bengaluru, 560048, India. Contact him at varun.agrawal@amd.com.

SAURABH GUPTA is a silicon design engineer at Advanced Micro Devices, Bengaluru, 560048, India. Contact him at saurabh.gupta@amd.com.

ASHISH SIRASAO is a corporate vice president of engineering at Advanced Micro Devices, Santa Clara, CA, 95054, USA. Contact him at ashish.sirasao@amd.com.

ELLIOTT DELAYE is a senior fellow software development engineer at Advanced Micro Devices, Santa Clara, CA, 95054, USA. Contact him at elliott.delaye@amd.com.

STEVE REINHARDT is a senior fellow software development engineer at Advanced Micro Devices, Redmond, WA, 98052, USA. Contact him at steve.reinhardt@amd.com.

RAJESH VIVEKANANDHAM is a systems design engineer at Advanced Micro Devices, Bengaluru, 560048, India. Contact him at rajesh.vivekanandham@amd.com.

RALPH WITTIG is a corporate fellow at Advanced Micro Devices, San Jose, CA, 95124, USA. Contact him at ralph.wittig@amd.com.

VINOD KATHAIL is a senior fellow engineer at Advanced Micro Devices, San Jose, CA, 95124, USA. Contact him at vinod.kathail@amd.com.

PADMINI GOPALAKRISHNAN is a senior director of software development at AMD, Hyderabad, 500081, India. Contact her at padmini.gopalakrishnan@amd.com.

SATYAPRAKASH PAREEK is a software development engineer at Advanced Micro Devices, San Jose, CA, 95124, USA. Contact him at satyaprakash.pareek@amd.com.

RISHABH JAIN is a Ph.D. student in the Computer Science and Engineering Department, Pennsylvania State University, University Park, PA, 16802, USA. Contact him at rishabh@psu.edu.

MAHMUT TAYLAN KANDEMIR is a distinguished professor of computer science and engineering at Pennsylvania State University, University Park, PA, 16802, USA. Contact him at mtk2@psu.edu.

JUN-LIANG LIN is a Ph.D. student in the Computer Science and Engineering Department, Pennsylvania State University, University Park, PA, 16802, USA. Contact him at jpl6521@psu.edu.

GULSUM GUDUKBAY AKBULUT is a Ph.D. student at the Computer Science and Engineering Department, Pennsylvania State University, University Park, PA, 16802, USA. Contact her at gulsum@psu.edu.

CHITA R. DAS is a distinguished professor of computer science and engineering at Pennsylvania State University, University Park, PA, 16802, USA. Contact him at cxd12@psu.edu.